JavaScript is disabled on your browser.

compact1, compact2, compact3

java.text

Class Normalizer

java.lang.Object
    java.text.Normalizer

```
public final class Normalizer
extends Object
```

This class provides the method `normalize` which transforms Unicode text into an equivalent composed or decomposed form, allowing for easier sorting and searching of text. The `normalize` method supports the standard normalization forms described in Unicode Standard Annex #15 — Unicode Normalization Forms.

Characters with accents or other adornments can be encoded in several different ways in Unicode. For example, take the character A-acute. In Unicode, this can be encoded as a single character (the "composed" form):

```
U+00C1    LATIN CAPITAL LETTER A WITH ACUTE
```

or as two separate characters (the "decomposed" form):

```
U+0041    LATIN CAPITAL LETTER A
U+0301    COMBINING ACUTE ACCENT
```

To a user of your program, however, both of these sequences should be treated as the same "user-level" character "A with acute accent". When you are searching or comparing text, you must ensure that these two sequences are treated as equivalent. In addition, you must handle characters with more than one accent. Sometimes the order of a character's combining accents is significant, while in other cases accent sequences in different orders are really equivalent.

Similarly, the string "ffi" can be encoded as three separate letters:

```
U+0066    LATIN SMALL LETTER F
U+0066    LATIN SMALL LETTER F
U+0069    LATIN SMALL LETTER I
```

or as the single character

```
      U+FB03    LATIN SMALL LIGATURE FFI
```

The ffi ligature is not a distinct semantic character, and strictly speaking it shouldn't be in Unicode at all, but it was included for compatibility with existing character sets that already provided it. The Unicode standard identifies such characters by giving them "compatibility" decompositions into the corresponding semantic characters. When sorting and searching, you will often want to use these mappings.

The `normalize` method helps solve these problems by transforming text into the canonical composed and decomposed forms as shown in the first example above. In addition, you can have it perform compatibility decompositions so that you can treat compatibility characters the same as their equivalents. Finally, the `normalize` method rearranges accents into the proper canonical order, so that you do not have to worry about accent rearrangement on your own.

The W3C generally recommends to exchange texts in NFC. Note also that most legacy character encodings use only precomposed forms and often do not encode any combining marks by themselves. For conversion to such character encodings the Unicode text needs to be normalized to NFC. For more usage examples, see the Unicode Standard Annex.

Since:

1.6

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static class	Normalizer.Form	This enum provides constants of the four Unicode normalization forms that are described in **Unicode Standard Annex #15 — Unicode Normalization Forms** and two methods to access them.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static boolean	isNormalized(CharSequence src, Normalizer.Form form)	Determines if the given sequence of

		char values is normalized.
static String	**normalize**(**CharSequence** src, **Normalizer.Form** form)	Normalize a sequence of char values.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

normalize

```
public static String normalize(CharSequence src,
                               Normalizer.Form form)
```

Normalize a sequence of char values. The sequence will be normalized according to the specified normalization from.

Parameters:

src - The sequence of char values to normalize.

form - The normalization form; one of Normalizer.Form.NFC, Normalizer.Form.NFD, Normalizer.Form.NFKC, Normalizer.Form.NFKD

Returns:

The normalized String

Throws:

NullPointerException - If src or form is null.

isNormalized

```
public static boolean isNormalized(CharSequence src,
                                   Normalizer.Form form)
```

Determines if the given sequence of char values is normalized.

Parameters:

src - The sequence of char values to be checked.

form - The normalization form; one of Normalizer.Form.NFC, Normalizer.Form.NFD, Normalizer.Form.NFKC, Normalizer.Form.NFKD

Returns:

true if the sequence of char values is normalized; false otherwise.

Throws:

`NullPointerException` - If src or form is null.

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.